1. Complexity analysis

| | | Best | Worst |
|---|---|---|---|
| BST | Insertion | O(1) | O(n) |
| | Serch | O(1) | O(n) |
| | Deletion | O(1) | O(n) |

Best case for BST operations are O(1)

Insertion → tree is balanced and insertion operation is left.

Searching → searched value is in the root

Deleting → deleting only root or only child

Worst cases:

Insert → Every existing node is visited and you need to visit deepest level

Search → Value that doesn't exist

Delete → Every node should be visited.

| | | Best | Worst |
|---|---|---|---|
| Hash Table | Insertion | O(1) | O(n) |
| | Deletion | O(1) | O(n) |
| | Search | O(1) | O(n) |

Сканировано с CamScanner

Best cases

Insertion → inserting into empty Bucket

Search → requested value is the first value in the Bucket

Delete → requested value is the only or the first value in the Bucket

Woust cases

Insertion → existing keys occupy all n spots at the Bucket.

Deletion → All n keys collide

Search → All n keys collide or value doesn't exist.

## 2. Experiment results

Benchmark uses 100, 1000, 10000 uniquely generated values from seed 63. Results can be seen in the repository's screenshots folder.

Hash table was fast for deletion and searching. and was better at larger inputs. For example, Hash table was approx.

* x8-4 Faster in the serch and deletion

Сканировано с CamScanner

# Conclusion

In conclusion, hash table was fosten than the BST because of <key, value> accessing. Perfomaree depends on many factors suchas, hash collisions, memory allocation, JVM, tree shape and etc. Both data structures have their own cons and pros.
Hash table is suitable for key lookup, while BST can be useful for sorted traversal